Mathematical Modeling Lecture Notes
Chapter 5 – Simulation

I. Introduction
   a. Simulation involves imitating the performance of some physical system.
   b. It is used to obtain some understanding of the system when the experimental approach is impossible or is too intractable or expensive to be of practical value.
      i. My dissertation research: Arrival of loads and trucks over time (intractable)
      ii. Stress on a new bridge (expensive)
      iii. The effect of a large meteor hitting the earth (impossible)
   c. We normally perform a large number of simulations, and then we gather statistics from these.
   d. This kind of simulation is sometimes called *Monte Carlo simulation*.

II. Simulate the sum of 10 consecutive rolls of a fair die.
   a. Generating random variables, first pass
      i. To do the simulation we'll need to be able to generate an instance of a random variable with a discrete uniform $\{1, 2, 3, 4, 5, 6\}$ distribution.
      ii. Fortunately, most software programs have built-in functions for generating discrete uniform random variables.
      iii. Show Python code.
   b. We can also use simulation to estimate a probability distribution.
      i. Show histogram.
      ii. Show mean, standard deviation, and maximum.

III. Generating random variables: basics
   a. This is one of the big things to worry about in simulation.
      - Many software applications (including Python) have built-in methods for generating random numbers from different distributions, but it's worth knowing how to do this yourself.
   b. Methods for generating numbers from a uniform distribution
      i. Strictly speaking, numbers produced by a computer aren't random; they're completely predictable.
         1. They do, however, appear to be random.
         2. They're sometimes called *pseudo-random numbers* as a result.
      ii. Middle square method
         1. This method works like so:
            - Start with a four-digit number x_0, called the *seed*.
            - Square it to obtain an eight-digit number (add a leading zero if necessary).
            - Take the middle four digits as the next random number.
         2. The sequence thus produced appears to be random over the interval 0 to 9999.
         3. It can be scaled if desired.
         4. Do example with Python.
         5. Problem: This method has a tendency to degenerate to zero, at which point it will remain zero forever.
         6. You can use more than four digits, but always take a middle number of digits equal to the number of digits in the seed.

iii. Linear congruential method
   1. A common method for generating random numbers involves linear congruences.
   2. Given c, the *linear congruential method* generates a sequence of random integers between 0 and $c-1$.
   3. The number x_{n+1} in the sequence is given by $x_{n+1} = (ax_n + b) \pmod{c}$.
      - The values of a, b, and c are given.
      - The value of x_0, often supplied by the user, is called the *seed*.
      - This is modular arithmetic; x_{n+1} is the remainder when ax_n+b is divided by c.
   4. Generate random numbers for students using Python.
   5. As you can see, after some number of integers you start cycling.
      - Sometimes you get all the integers between 0 and $c-1$, and sometimes you don't.
      - However, eventually you end up in a cycle. How long is the cycle?
      - Theorem: The linear congruential sequence defined by $x_{n+1} = (ax_n + b) \pmod{c}$ has period length c if and only if
        - b is relatively prime to c;
        - $a-1$ is a multiple of p for every prime p dividing c;
        - if c is a multiple of 4, then $a-1$ is a multiple of 4.
      - Proof: See Knuth, *The Art of Computer Programming*, Vol. 2, Section 3.2.

iv. Python's implementation
   1. Python's `numpy.random` library uses a collection of more sophisticated methods.
   2. However, these can give you a sense of how a computer might generate random numbers.

v. Generating continuous uniform random numbers
   1. Most modern random number generator libraries will do this directly.
      - For example, after creating an instance of `np.random.default_rng()`, you can call `rng`'s `uniform` function to generate a continuous uniform random number.
   2. What if you don't have a way to do this directly?
      - A common method of generating a continuous uniform random number is the following: $u_n = (x_n + 1/2)/c$, where $\{x_n\}$ is the sequence of numbers obtained by the linear congruential method.
      - If c is large (say, on the order of 2^{191}), and the linear congruential sequence has cycle length c, then this will do a decent job of producing a number from a continuous uniform $(0, 1)$ distribution.
      - To obtain a number from the continuous (a, b) distribution, apply the transformation $a + u_n(b - a)$.

vi. Generating a sequence of uniform random integers
   1. You don't want to use the immediate results from the linear congruential method if you're generating a sequence of uniform random integers.
   2. This is because the sequence isn't really random; you go through the entire set of numbers in the cycle before a repeat occurs.

3. To obtain a sequence of random integers in the range $\{a, a+1, \ldots, b\}$ you could find a uniform random number in the range $(a, b+1)$ using the method just discussed and round down.

IV. Brief primer on probability
   a. Before we discuss how to generate more complicated random variables we're going to need to review some basic probability theory.
   b. There are two basic types of random variables.
      i. *Discrete random variables* take outcomes from a discrete probability space.
         1. Examples
            - Let X be the sum when two fair dice are tossed.
            - Toss a fair die repeatedly. Let X be the number of tosses until a six appears.
            - Neither of these random variables is uniform; that, is, not all outcomes are equally likely.
         2. Discrete random variables are generally easier to simulate than continuous random variables.
      ii. *Continuous random variables* take outcomes from a continuous probability space.
         1. Examples
            - Let X be the amount of time you spend waiting for the bus the next time you arrive at a bus stop.
            - Let X be the height of the next person who walks through the door of the classroom.
            - Neither of these random variables is uniform, either.
         2. Continuous random variables are a little trickier to simulate.
            - For example, how would you simulate a set of waiting times for a bus if they're not all equally likely?
            - Even worse, what does equally likely even mean when there are infinitely many possible outcomes?
   c. Probability and continuous random variables
      i. There is no way to assign positive probabilities to uncountably many individual outcomes and have them all add up to 1.
      ii. Instead of assigning probabilities to individual outcomes, then, we assign probabilities to intervals of outcome values.
      iii. We do this by means of areas, so that the area under a curve that corresponds to an interval is the probability that the random variable takes on a value in that interval.

      iv. There are two functions that we use to keep track of these probabilities.
         1. The *probability density function* $f(x)$ is the function such that the area under it corresponds to a probability.
            - Thus $P(a \leq X \leq b) = \int_a^b f(x)\,dx$.
         2. The *cumulative distribution function* $F(x)$ is the function such that $F(a) = P(X \leq a)$.

- Thus $P(a \le X \le b) = F(b) - F(a)$.
- Also, $F(a) = \int_{-\infty}^{a} f(x)\,dx$.

v. If we know either of these two functions for a continuous random variable X then we have a good understanding of how X works, probabilistically.

V. Generating random numbers from other distributions

a. Probability integral transform

i. There is a method for using uniformly distributed random numbers to generate a random number from any distribution.

ii. Here's the idea

1. Suppose $X \sim U(0,1)$, and $Y \sim N(\mu, \sigma^2)$.
2. We want a function to map from X to Y.
3. One way to do that is to imagine mapping areas to areas, like so:

4. We want to associate the values of X and of Y that give the same cdf value.
5. So, in order to obtain this, we need the relationship $F(Y) = G(X) = X$, where F is the cdf of Y and G is the cdf of X.
6. So we need to have $Y = F^{-1}(X)$.

iii. For some functions F^{-1} is fairly simple to determine analytically, and for others you would need to do a numerical approximation.

iv. Example: the exponential distribution

1. The pdf of the exponential distribution is $f(x) = \lambda e^{-\lambda x}$, $x \ge 0$.
2. The cdf is $F(x) = \int_0^x \lambda e^{-\lambda t} dt = 1 - e^{-\lambda x}$.
3. Solve for x: $F(x) - 1 = -e^{-\lambda x} \implies -\lambda x = \ln(1 - F(x)) \implies F^{-1}(x) = -\ln(1-x)/\lambda$.
4. Thus if x is an observation from a $U(0,1)$ distribution, then $y = -\ln(1-x)/\lambda$ is an observation from an *exponential*(λ) distribution.
5. Moreover, x having a $U(0,1)$ distribution is equivalent to $1 - x$ having a $U(0,1)$ distribution, so we often express the transformation as $y = -\ln x/\lambda$.
6. Show students Python simulation in `ProbIntegralTransform.ipynb` file.

b. Normal

i. The normal cdf is nasty to invert, as it doesn't even have a closed-form expression: $F(x) = \displaystyle\int_{-\infty}^{x} \frac{e^{-(t-\mu)^2/(2\sigma^2)}}{\sqrt{2\pi}\sigma}\,dt$.

ii. However, if U_1 and U_2 are independent uniform(0,1) random variables, then the following Box-Muller transform generates two independent $N(0,1)$ random variables:

$$Z_1 = \sqrt{-2\ln U_1}\cos(2\pi U_2),$$
$$Z_2 = \sqrt{-2\ln U_1}\sin(2\pi U_2).$$

iii. Show students Python simulation in `BoxMuller.ipynb` file.

iv. To get a $N(\mu, \sigma^2)$ random variable, use the transformation $Y = \sigma Z + \mu$, where $Z \sim N(0,1)$.

c. Acceptance-rejection method
   i. This method involves the following procedure for generating observations from a distribution with pdf $f(x)$.
      1. Generate a uniform random number x on the support of the distribution.
         - If the support is infinite then choose some finite interval with probability almost 1 that a random variable from that distribution takes on a value in that interval.
      2. Then decide whether to accept x.
         - Accept x with probability $f(x)/\max f(x)$.
         - To do that generate a uniform random number r on the interval $(0, \max f(x))$.
         - If $r \leq f(x)$ then accept x.
         - If $r > f(x)$ then reject x.
   ii. The distribution of accepted values has $f(x)$ as its probability density function.
      1. To see this, let $[\alpha, \omega]$ be the interval on which the random variable takes its values.
      2. We have

$$\begin{aligned} P(X \leq a | X \text{ is accepted}) &= \frac{P(X \leq a \text{ and } X \text{ is accepted})}{P(X \text{ is accepted})} \\ &= \frac{\int_\alpha^a \frac{f(x)}{\max f(x)} \frac{dx}{\omega - \alpha}}{\int_\alpha^\omega \frac{f(x)}{\max f(x)} \frac{dx}{\omega - \alpha}} \\ &= \frac{\frac{1}{\max f(x)(\omega - \alpha)} \int_\alpha^a f(x)\, dx}{\frac{1}{\max f(x)(\omega - \alpha)} \int_\alpha^\omega f(x)\, dx} \\ &= \int_\alpha^a f(x)\, dx. \end{aligned}$$

   iii. Show students Python simulation in `AcceptReject.m` file.
   iv. The acceptance/rejection method is quite powerful because it works as long as you have $f(x)$.
      - However, if you don't pick your range carefully you can end up generating a bunch of values that aren't used.

VI. Roulette simulation
   a. Rules
      i. There are 38 spaces on a roulette wheel: 0, 00, and 1-36.
      ii. Half the spaces numbered 1-36 are red, and the other half are black.
      iii. The two spaces 0 and 00 are green.
   b. First simulation (do in class)
      i. Simulate betting red or black, which pay 1:1.
      ii. Bet \$1 per game, and keep track of the earnings.
      iii. Determine winnings per game, longest winning streak, and longest losing streak. (Show for a small number of games first, then for a large number.)
      iv. On average, the winnings will be about -0.053 dollars per game. (Do this calculation: $1(18/38) - 1(20/38)$.)
      v. The fact that the sample mean converges to the theoretical mean is a consequence of the law of large numbers.
   c. Second simulation (do in class)
      i. Simulate betting green, which pays 17:1.

ii. Bet $1 per game, and keep track of the earnings.
iii. Determine winnings per game, longest winning streak, and longest losing streak. (Show for a small number of games first, then for a large number.)
iv. As before, the average winnings will be about -0.053 dollars per game. (Do this calculation: $17(2/38) - 1(36/38)$.)
v. The winning streaks are a lot shorter, and the losing streaks are a lot longer.

d. Comparison: Betting on green is more likely to produce both a big win and a big loss.

VII. Craps simulation
a. The game of craps works like this:
i. You roll two dice.
1. If the sum of the dice is 7 or 11 on the first roll, then you win.
2. If the sum of the dice is 2, 3, or 12 on the first roll, then you lose.
3. Otherwise, the number you roll becomes your *point*.
ii. If you haven't won or lost yet then you continue to roll until you either
1. Roll a 7, in which case you lose, or
2. Roll your point, in which case you win.

b. What's the probability of winning a game of craps?
i. Because of the complexity of the game, a straightforward analysis is going to be rather tricky (although I did assign this problem, with some hints, one of the last times I taught probability theory).
ii. However, if we can simulate a large number of games of craps we can use the simulations to find the probability of winning.

c. Simulate this in Python (in class) and show results to students.
d. The probability of winning at craps is about 0.493.

Homework:

- 5.1 #1, 6 (Actually implement these in Python.)
- 5.2 #1, 2
- 5.3 #2, 4
- The Weibull distribution has pdf $f(t) = \begin{cases} 0, & t \leq \nu; \\ \frac{\beta}{\alpha}\left(\frac{t-\nu}{\alpha}\right)^{\beta-1} \exp\left(-\left(\frac{t-\nu}{\alpha}\right)^{\beta}\right), & t > \nu. \end{cases}$ Here, $\nu, \alpha,$ and β are parameters. (The Weibull distribution is used to model the lifetime of an object featuring something like a "weakest link.") Use the probability integral transform technique to find a formula $y = g(x)$ such that if X is a uniformly distributed random variable on $(0, 1)$, then $Y = g(X)$ is a Weibull-distributed random variable.
- Use the acceptance-rejection method to generate 1000 values from a $\chi^2(4)$ probability distribution. Use a range of $[0, 15]$ for the input values. The $\chi^2(4)$ pdf is $f(t) = \dfrac{te^{-t/2}}{4}$. Use Python to plot these values in a histogram. Then compare the histogram with a plot of $f(t)$.